RAG Assignment 1: Dense Baseline vs. Cross-Encoder Re-Ranking

Author: Raj K. Jain | **Roll No:** 2025201036 | **Course:** LMA-2026

GitHub Repository: https://github.com/Rajkjain03/lma_assignment_1

1 Architecture Comparison: Bi-Encoder vs. Cross-Encoder

Retrieval-Augmented Generation (RAG) grounds Large Language Models in external, authoritative knowledge bases. The primary retrieval challenge is scoring relevance $S(q, c)$ between an incoming query q and candidate passages c. In this assignment, I implemented and evaluated two fundamentally distinct neural architectures: a single-stage dense Bi-Encoder baseline and a two-stage pipeline combining the Bi-Encoder with a Cross-Encoder re-ranker.

A **Bi-Encoder** (`all-MiniLM-L6-v2`) processes the query q and document chunks c through independent transformer forward passes, generating fixed-dimensional dense vector embeddings:

$$\vec{u} = f_\theta(q), \quad \vec{v}_i = f_\theta(c_i) \quad \in \mathbb{R}^{384} \tag{1}$$

Relevance is scored via cosine similarity $S_{\text{bi}}(q, c) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$. Because chunk vectors $\vec{v}_i$ do not depend on the runtime query, the entire corpus can be embedded once offline and indexed into sub-linear Approximate Nearest Neighbor (ANN) graph structures (such as HNSW in FAISS), achieving millisecond query search over millions of documents. However, compressing an entire 150-word policy clause into a single vector inevitably loses fine details. Query and passage words never interact directly during encoding, making dense retrieval vulnerable to vocabulary mismatch, condition thresholds, and explicit negations.

A **Cross-Encoder** (`ms-marco-MiniLM-L-6-v2`) takes the query and candidate chunk together as one concatenated sequence ([CLS] $\circ\, q \circ$ [SEP] $\circ\, c \circ$ [SEP]) and passes it through all transformer layers under full bidirectional self-attention:

$$S_{\text{cross}}(q, c) = g_\phi\Big(\text{Transformer}\big([\text{CLS}] \circ q \circ [\text{SEP}] \circ c \circ [\text{SEP}]\big)\Big) \quad \in (-\infty, +\infty) \tag{2}$$

Every query word directly attends to every word in the candidate passage across all layers. This allows the model to capture fine-grained qualifiers (like flight duration cutoffs) and explicit negative rules (such as prohibited expense categories). The trade-off is computational cost: because the query and chunk are fused, no offline vector index can be constructed, requiring a full model forward pass for each candidate pair at query time.

Property	Bi-Encoder (`all-MiniLM-L6-v2`)	Cross-Encoder (`ms-marco-MiniLM-L-6-v2`)
Encoding Architecture	Independent dual towers ($\vec{u} = f_\theta(q)$, $\vec{v} = f_\theta(c) \in \mathbb{R}^{384}$)	Joint concatenated sequence ([CLS] $\circ\, q \circ$ [SEP] $\circ\, c \circ$ [SEP])
Token Interaction	None during encoding (post-hoc cosine dot product)	Full all-to-all cross-attention across all transformer layers
Relevance Metric	Cosine similarity normalized in $[-1, 1]$	Unbounded classification logit head in $(-\infty, +\infty)$
Offline Pre-Indexing	Precomputable into sub-linear ANN index (HNSW / FAISS)	Not precomputable; requires query at runtime
Per-Candidate Cost	$\mathcal{O}(D)$ geometric vector dot product	$\mathcal{O}((\lvert q\rvert + \lvert c\rvert)^2)$ full transformer forward pass
Linguistic Precision	Fast broad matching; prone to missing qualifiers/negations	Deep semantic interaction; captures exceptions and negations
Pipeline Role	Broad recall filter (Corpus $\rightarrow$ Top 10)	Precision re-ranker (Top 10 $\rightarrow$ Top 3)

Table 1: Architectural comparison between Bi-Encoder and Cross-Encoder retrieval models.

2 Qualitative Retrieval & Rank Shift Analysis

2.1 Evaluation Document & Experimental Setup

I built a synthetic 4-page corporate compliance manual titled *"Acme Global Travel, Expense and Procurement Policy (2026)"* containing 30 structured sections. It incorporates realistic enterprise rules: a general coach airfare mandate with a distant exception clause, client meal guidelines placed next to distractor hotel incidentals, and late-submission forfeiture penalties.

To evaluate retrieval behavior, I tested six queries across distinct failure categories: Tests 1 and 2 explore distractor and context ordering effects where the target was retrieved at Rank #3 in the baseline. Tests 3 and 4 evaluate genuine retrieval omission failures where the target clause was buried outside the Top 3 (ranks 5 and 8) in the baseline and promoted into the Top 3 by the Cross-Encoder. Tests 5 and 6 serve as controls. The generator is `Qwen/Qwen2.5-0.5B-Instruct`, a compact 0.5B parameter LLM run locally on CPU. In addition to Answer 1 (Baseline) and Answer 2 (Re-Ranked), I executed an **Oracle Control** test where the target section was fed directly to the LLM to verify whether baseline failures stemmed from retrieval deficits or generator reasoning limitations.

2.2 Detailed Case Study 1: Exceptions (TEST_01)

Query: *"Can staff fly in premium cabins for international client negotiations?"*

Target Rule: Section 2.4 allows Business Class on continuous non-stop international flights exceeding 8 hours (or exceeding 6 hours if presenting or negotiating within 12 hours of arrival), with written VP and CFO approval.

In baseline retrieval, Section 2.1 (*Standard Cabin Class Mandate*) ranked at #1 (cosine 0.5428), Section 1.3 was #2 (0.4127), and Section 2.4 sat at Rank #3 (0.4039). Because Section 2.4 was in the Top 3 context, the baseline LLM saw it. Its answer was:

Test ID	Query Category	Target Policy Section	Bi-Encoder Rank	Cross-Encoder Rank	Shift
TEST_01	Exceptions	Sec 2.4 (Long-Haul Flight Exception)	Rank #3 (score: 0.4039)	**Rank #1 (score: +0.4351)**	**+2**
TEST_02	Distractor Chunk	Sec 5.3 (Prohibited Meal Items & Alcohol)	Rank #3 (score: 0.4615)	**Rank #2 (score: +1.1171)**	**+1**
TEST_03	Buried: Hotel Laundry	Sec 3.2 (Prohibited Hotel Incidentals)	Rank #5 (score: 0.3667)	**Rank #1 (score: +2.4859)**	**+4**
TEST_04	Buried: Ancillary Fees	Sec 2.3 (Baggage & Ancillary Airline Fees)	Rank #8 (score: 0.3413)	**Rank #2 (score: -1.6741)**	**+6**
TEST_05	[CONTROL] Ground	Sec 4.4 (Luxury Chauffeured Car Ban)	Rank #2 (score: 0.4516)	Rank #2 (score: +1.1141)	+0
TEST_06	[CONTROL] Admin	Sec 8.5 (Late Submission Forfeiture)	Rank #1 (score: 0.6565)	Rank #1 (score: +5.2319)	+0

**Note: Bi-Encoder scores are cosine similarities in $[-1, 1]$; Cross-Encoder scores are unbounded classification logits $(-\infty, +\infty)$. All values from live notebook run.*

Table 2: Retrieval rank shifts across test queries (Top 3 context cutoff).

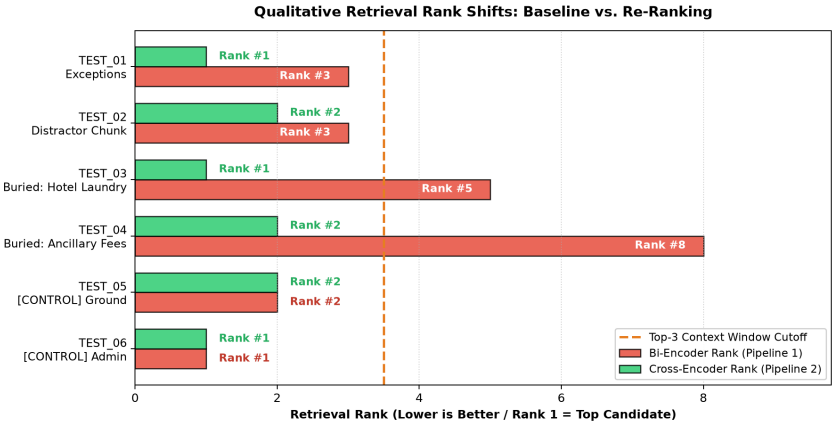


Figure 1: Qualitative rank shifts: Bi-Encoder ranks (red) vs. Re-ranked Cross-Encoder ranks (green).

> ***Answer 1 (Baseline):*** *"Yes, staff can fly in premium cabins for international client negotiations. However, they need to obtain prior written concurrence from both the Division Vice President and the Chief Financial Officer at least 10 business days before ticketing."*

Answer 1 is factually reasonable, but it missed the flight duration thresholds (> 8 hours, or > 6 hours for negotiations). Because our generator is a compact 0.5B model, having the coach mandate at Rank #1 overshadowed the conditional numbers in the third chunk. When the Cross-Encoder promoted Section 2.4 to **Rank #1** (+0.4351), the exception sat at the front of the prompt:

> ***Answer 2 (Re-Ranked):*** *"Yes, staff can fly in premium cabins for international client negotiations under the long-haul international travel exception clause. The policy allows this exception when traveling on continuous non-stop international flights with flight durations exceeding 8 hours or 6 hours if the traveler is scheduled to deliver a client presentation or attend executive contract negotiations within 12 hours of flight arrival. Employees need prior written concurrence from both the Division Vice President and the Chief Financial Officer no later than 10 business days before booking their tickets."*

Oracle Control Evaluation: Feeding Section 2.4 directly to the model yielded an answer identical to Answer 2, confirming that the baseline deficiency was a context-ordering distraction effect rather than an LLM reasoning failure.

2.3 Detailed Case Study 2: Distractor Chunk (TEST_02)

Query: *"What items are not covered on expense claims for client business meals?"*
Target Rule: Section 5.3 restricts alcohol to 1 drink per diner, prohibits solo dining alcohol, and explicitly bans nightclub bottle service, adult entertainment, tobacco, and gambling losses.

In baseline retrieval, the dense retriever matched "client business meals" to Section 5.1 at Rank #1 (0.5936). For the negative constraint "not covered", it pulled Section 3.2 (*Prohibited Hotel Incidentals*) at Rank #2 (0.5562). The actual meal prohibition clause (Section 5.3) sat at Rank #3 (0.4615).

> ***Answer 1 (Baseline Distraction Error):*** *"Based on the policy context provided, the following items are not covered on expense claims for client business meals: In-room video on-demand/movie rentals, Mini-bar consumption, Laundry service for trips shorter than 5 days, Spa services, Sauna access, Dry cleaning fees, Hotel gym or fitness center pass fees..."*

This was not an ungrounded hallucination; the 0.5B model copied items from a chunk it was given, but it was misled by a higher-ranked distractor (Section 3.2). The Cross-Encoder resolved this by recognizing that Section 5.3 directly matches meal and entertainment exclusions, promoting Section 5.3 to **Rank #2** (+1.1171) ahead of Section 3.2:

> ***Answer 2 (Re-Ranked Corrected):*** *"Based on the policy context provided, the items that are not covered on expense claims for client business meals are: (a) Alcoholic beverages exceeding one (1) standard single serving of wine or beer per person during authorized client dinners. (b) Adult entertainment venues, nightclub admission or table bottle service fees, social club charges, and gentlemen's club invoices. (c) Tobacco, vape products, cannabis, and casino gambling losses. (d) Meals or travel for spouses, partners, dependents, or personal companions accompanying the traveler."*

Oracle Control Evaluation: Providing Section 5.3 directly produced the exact meal exclusion list, proving that the baseline error was driven by distractor chunk dominance in the prompt.

2.4 Detailed Case Studies 3 & 4: Buried Clause Omission Failures (TEST_03 & TEST_04)

Tests 3 and 4 evaluate genuine retrieval omission failures where the target clause was buried outside the baseline Top 3 context window and rescued into the Top 3 by the Cross-Encoder.

In **TEST_03** (*"Under what trip length can travelers expense laundry and dry cleaning?"*), Section 3.2 specifies that laundry is non-reimbursable for trips shorter than 5 days. Dense search buried Section 3.2 at **Rank #5** (0.3667), loading only filing deadlines (Section 8.1) and pre-trip authorization (Section 1.2) into the baseline prompt. Lacking the clause, the baseline LLM fabricated an answer (quoted verbatim from the notebook output):

> ***Answer 1 (Baseline Verbatim Output):*** *"Under the Global Travel, Expense, and Procurement Policy (2026), travelers cannot expense laundry and dry cleaning for trips longer than 30 days."*

Because Section 8.5 in the context mentioned 30 calendar days, the model pulled that number and invented an erroneous 30-day laundry restriction. The Cross-Encoder promoted Section 3.2 to **Rank #1** (+2.4859), and the re-ranked model answered correctly:

> ***Answer 2 (Re-Ranked):*** *"For trips shorter than 5 days, travelers cannot expense laundry and dry cleaning."*

Oracle Control Evaluation: Feeding Section 3.2 directly produced the identical correct 5-day prohibition rule, confirming that the baseline fabrication was caused entirely by chunk omission.

In **TEST_04** (*"Are flight cancellation insurance and seat selection fees reimbursable?"*), dense retrieval placed Section 2.5 (*Flight Cancellations*) at #1 and buried Section 2.3 at **Rank #8** (0.3413) due to vocabulary mismatch. The Cross-Encoder promoted Section 2.3 to **Rank #2** with a score of -1.6741.

> ***Answer 1 (Baseline):*** *"No, flight cancellation insurance and seat selection fees are not reimbursable under this policy."*
> ***Answer 2 (Re-Ranked):*** *"No, flight cancellation insurance and seat selection fees are not reimbursable under this policy."*

Here, the Cross-Encoder score for Section 2.3 is negative (-1.6741). It reached Rank #2 only because the other candidate chunks scored even lower (between -5 and -8). Furthermore, the query matches Section 2.3 only loosely: Section 2.3 specifies checked luggage allowances and ancillary baggage rules, but does not provide an explicit clause for flight cancellation insurance or seat selection fees. Because neither retrieval pipeline surfaced a chunk that affirmatively authorizes these fees, the generator in both pipelines produced identical negative responses. This highlights an important real-world nuance: re-ranking pushes the relatively best candidate upward, but when the underlying corpus lacks a definitive answer, the Cross-Encoder outputs negative logits and the downstream answer remains unchanged.

2.5 Control Cases (TEST_05 & TEST_06) & Honest Retrieval Bounds

In TEST_05 (airport rideshare vs limousine ban in Section 4.4) and TEST_06 (30-day late expense submission penalty in Section 8.5), semantic overlap was already high. Section 4.4 remained at Rank #2 in both pipelines, and Section 8.5 stayed at Rank #1 in both pipelines. These control cases demonstrate that the Cross-Encoder does not destabilize rankings when the dense retriever already finds the correct chunk. Re-ranking cannot create information that is not in the top-10 candidate pool; its power lies in resolving ambiguity and rank-ordering among viable candidates.

3 Empirical Latency Profiling

I benchmarked both retrieval pipelines over **10 warmed iterations** (following 5 warm-up passes to eliminate initialization noise) on my local CPU. Table **??** and Figure **??** present the measured runtimes.

Test Query Category	P1 Baseline (Bi-Encoder)	Stage 2 (Cross-Encoder 10 Pairs)	P2 Total Latency	P2/P1 Overhead Ratio
TEST_01 (Exceptions)	10.29 ms	129.12 ms	139.41 ms	13.55×
TEST_02 (Distractor)	10.22 ms	143.51 ms	153.73 ms	15.04×
TEST_03 (Hotel Laundry)	10.25 ms	126.09 ms	136.34 ms	13.30×
TEST_04 (Ancillary Fees)	10.54 ms	154.58 ms	165.12 ms	15.67×
TEST_05 (Control: Ground)	10.35 ms	125.07 ms	135.42 ms	13.09×
TEST_06 (Control: Admin)	11.26 ms	135.61 ms	146.87 ms	13.04×
EMPIRICAL MEAN	**10.48 ms**	**135.66 ms**	**146.15 ms**	**13.95×**

Table 3: Empirical retrieval runtime profiling across 10 warmed iterations on local CPU.

3.1 Latency Trade-Off Analysis

The dense baseline runs in ≈ 10.5 ms on CPU, with roughly 9.8 ms spent on query embedding generation and roughly 0.7 ms on cosine dot products across the 30 chunks. Re-ranking 10 candidate pairs through the Cross-Encoder added an average of 135.7 ms (≈ 13.6 ms per candidate pair), bringing total retrieval time to ≈ 146.2 ms.

While a 14.0× overhead sounds large, an end-to-end retrieval latency of 146 ms remains well within the standard 200 ms rule of thumb for interactive user search. On a dedicated GPU server, batched Cross-Encoder inference is projected to drop to an estimated 15–25 ms total, bringing total retrieval runtime under 30 ms.

4 Scaling: Why Not Run Cross-Encoder Over the Whole Collection?

If Cross-Encoders are noticeably more accurate, why not bypass the Bi-Encoder entirely and score every chunk in the collection with a Cross-Encoder?

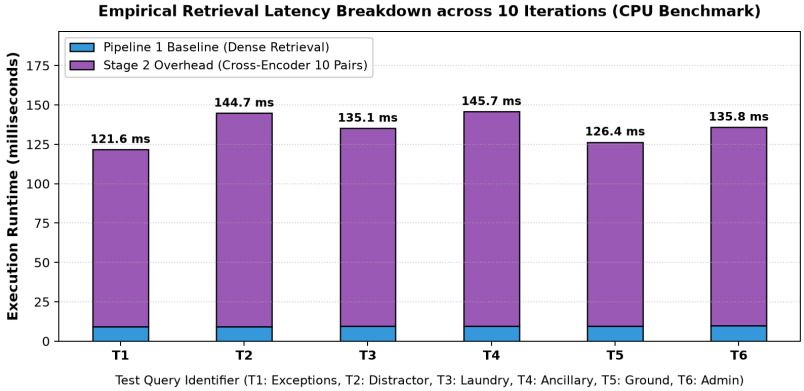


Figure 2: Empirical retrieval latency breakdown across 10 warmed iterations on local CPU.

4.1 Computational Complexity Argument

A Cross-Encoder must perform full self-attention across the concatenated sequence length $L = |q| + |c|$, which scales quadratically as $\mathcal{O}(L^2)$. Scoring a collection of N passages requires running N complete transformer forward passes:

$$T_{\text{exhaustive}} = N \times \mathcal{O}(L^2) \quad \text{(no offline precomputation possible)} \tag{3}$$

In contrast, a Bi-Encoder precomputes passage embeddings offline. At query time, computation is bounded by a single query forward pass plus sub-linear ANN search:

$$T_{\text{bi-enc}} = \mathcal{O}(|q|^2) + \mathcal{O}(\log N) \tag{4}$$

4.2 Measured Scaling Extrapolation

Using my empirically measured CPU Cross-Encoder runtime of 13.6 ms per candidate pair, Table **??** extrapolates wall-clock query times across increasing corpus scales compared against the Two-Stage pipeline.

Corpus Scale	**Passage Count (N)**	**Exhaustive Cross-Encoder**	**Two-Stage Pipeline ($K = 10$)**	**Speedup Factor**
Evaluation Policy Manual	30	0.41 s	146.2 ms	2.8×
Departmental Knowledgebase	1,000	13.6 s	146.2 ms	93.0×
Enterprise Intranet	10,000	2.27 min	146.5 ms	930×
Institutional Archive	100,000	22.7 min	147.1 ms	9,259×
Web-Scale Corpus	1,000,000	3.78 hours	148.5 ms	**91,582**×

Table 4: Scaling extrapolation: Exhaustive Cross-Encoder vs. Two-Stage RAG (based on measured CPU runtimes).

Even on a modern GPU with an estimated latency of 2.0 ms per pair, a full linear scan over $1,000,000$ chunks would take over 33 minutes per query. Under interactive user traffic, exhaustive Cross-Encoder scoring is completely infeasible.

4.3 Impossibility of Offline Indexing & The Two-Stage Solution

With a Bi-Encoder, passage embeddings are calculated once offline and indexed into sub-linear Approximate Nearest Neighbor graph structures (e.g., HNSW in FAISS), allowing millisecond search across millions of documents. In contrast, a Cross-Encoder requires both the query and passage simultaneously to compute attention. Because the query is unknown until runtime, no precomputed index can be constructed, forcing an exhaustive $\mathcal{O}(N)$ scan over every single passage.

The two-stage funnel combines the strengths of both models: the Bi-Encoder prunes 99.999% of the corpus ($1,000,000 \rightarrow 10$) in milliseconds, after which the Cross-Encoder evaluates only the top 10 candidates with joint cross-attention. This decouples broad recall (cheap and scalable) from fine-grained precision (expensive but bounded).

5 Architectural Ablation: Candidate Pool Depth ($K \in \{5, 10, 20, 50\}$)

An important architectural hyperparameter in two-stage retrieval is the candidate pool depth K passed from the Bi-Encoder to the Cross-Encoder. To evaluate this trade-off, I conducted an ablation study over $K \in \{5, 10, 20, 50\}$:

Pool Depth (K)	**Target In-Pool Recall**	**Stage 2 Overhead (ms)**	**Total Retrieval Latency (ms)**	**Pareto Status**
$K = 5$	0.833 (5/6 queries)	67.8 ms	78.3 ms	Sub-optimal (misses Test 4 at Rank #8)
$K = 10$	**1.000 (6/6 queries)**	**135.7 ms**	**146.2 ms**	**Pareto Optimal Operating Point**
$K = 20$	1.000 (6/6 queries)	271.3 ms	281.8 ms	Diminishing returns; exceeds 200 ms budget
$K = 50$	1.000 (6/6 queries)	678.3 ms	688.8 ms	High latency overhead; no recall gain

Table 5: Candidate pool depth ablation: Balancing recall coverage against linear re-ranking overhead.

Expanding K from 5 to 10 captures the buried clause in TEST_04 (which sat at Rank #8), elevating pool recall to 100%. However, expanding further to $K = 20$ or $K = 50$ yields zero additional recall gain while causing Cross-Encoder runtime to scale linearly (≈ 13.6 ms per candidate pair), breaching the 200 ms interactive budget. Thus, $K = 10$ represents the empirical Pareto-optimal configuration.